A HANDS-ON GUIDE TO YOUR APP

Learn React by reading TaskSyncer

A step-by-step tour of the JavaScript and JSX frontend you already have

START	TRACE	PRACTICE
Follow the app from browser entry to page	See how state, API calls, and realtime fit together	Build a small search feature as a guided exercise

Made for the TaskSyncer frontend in this workspace

This guide describes the current files under **frontend/src**. It uses JavaScript and JSX examples, not TypeScript.

How to use this guide

Read in order once, then keep it beside the source code. Each lesson answers three questions: what React idea is being used, where it appears in TaskSyncer, and what to look at next.

Before you read the code

• Open the project in your editor and keep the **frontend** folder expanded.

• Start the frontend from a terminal in **frontend** with `npm start`. The backend must also be running for sign-in and project data.

• When this guide says a file path, it is relative to the repository root. For example, **frontend/src/App.jsx**.

• Do not try to memorize every line. First trace one action from a button to the server and back to the screen.

The simplest mental model
React renders the screen from data. When component state or props change, React runs the component again and updates the parts of the page that need to change.

A quick glossary

Word	Meaning in this app
Component	A function that returns a piece of interface, such as Navbar or TaskCard.
JSX	HTML-like syntax inside JavaScript, such as `<ProjectCard />`.
Props	Inputs passed from a parent component to a child component.
State	A component's remembered data that can change, such as an open modal or project list.
Effect	Code React runs to synchronize with something external, such as an API or timer.
Hook	A React function such as useState or useEffect that gives a component capabilities.

1. Find where React starts

A web app needs a starting file. TaskSyncer's startup path is short: HTML gives JavaScript a mount point, then React renders App into it.

IN YOUR PROJECT: **frontend/src/index.js** - browser entry: creates the React root and renders App.

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./index.css";

const root = createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Read it from the bottom up

• **document.getElementById('root')** finds the empty element in the HTML page.

• **createRoot(...)** tells React to manage that element.

• **root.render(...)** puts the App component on the page.

• **StrictMode** helps reveal some mistakes during development. It does not change the app's production purpose.

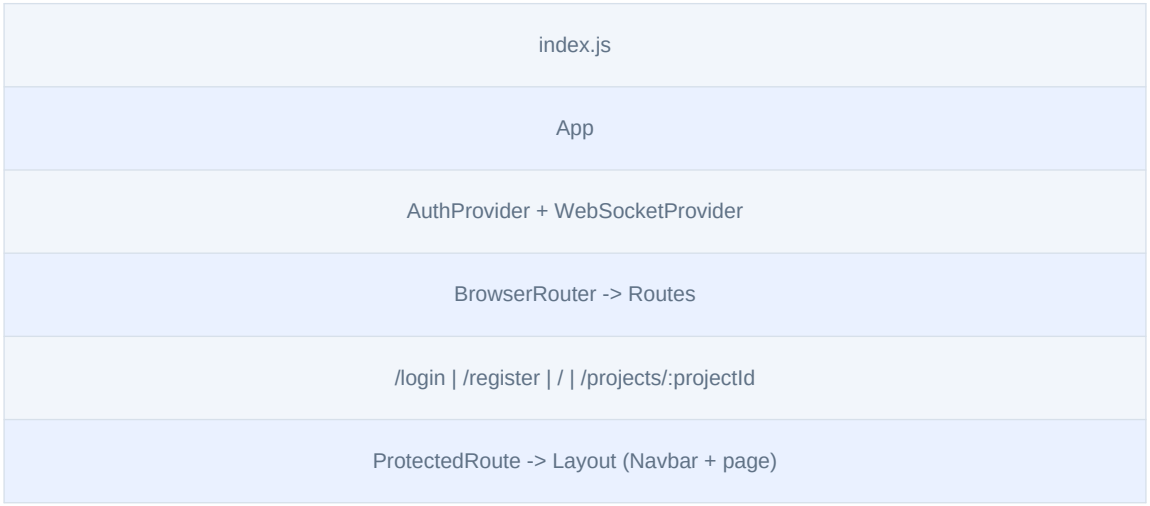
A component is a JavaScript function that returns JSX. The tag `<App />` asks React to run that component.

Try it
Open **frontend/public/index.html** and find the element with id **root**. Then compare it with the getElementById call above.

2. Follow the component tree

App.jsx connects app-wide providers, URLs, and page components. It is the map of where a screen comes from.

IN YOUR PROJECT: **frontend/src/App.jsx** - declares providers, routes, layout, and redirects.



Routes are URL-to-component rules

```
<Route path="/login" element={<Login />} />
<Route path="/register" element={<Register />} />
<Route path="/" element={
  <ProtectedRoute><Layout><Dashboard /></Layout></ProtectedRoute>
} />
<Route path="/projects/:projectId" element={
  <ProtectedRoute><Layout><ProjectView /></Layout></ProtectedRoute>
} />
```

The **:projectId** part is a route parameter. ProjectView reads it with `useParams()`. ProtectedRoute checks whether a user is signed in. Layout is shared page framing, so the navbar does not need to be re-created on each screen.

Trace it
Open Dashboard in the browser. Find the matching path in **App.jsx**, then the Dashboard import, then **pages/Dashboard.jsx**.

3. JSX, components, and props

Look at cards first: they are small enough to understand and show how a parent can reuse one component for many records.

IN YOUR PROJECT: **frontend/src/components/ProjectCard.jsx** - displays one project and receives project, isMember, and onDeleted.

```
export default function ProjectCard({ project, isMember, onDeleted }) {
  const isOwner = project.owner_id === user?.id;
  return (
    <div onClick={() => isMember && navigate(`/projects/${project.id}`)}>
      <h3>{project.name}</h3>
      {isMember && <span>{isOwner ? "Owner" : "Member"}</span>}
    </div>
  );
}
```

• **function ProjectCard(...)** defines a component. Capital letters matter: React treats capitalized JSX tags as components.

• The values inside the function are props from the parent.

• **{project.name}** inserts a JavaScript expression into JSX.

• **{condition && ...}** renders the right side only when the condition is true.

• The ternary expression picks one of two labels: Owner or Member.

JSX resembles HTML but has JavaScript rules

• Use `className` instead of HTML's class attribute.

• Close tags, including input tags: `<input />`.

• Use braces for JavaScript: `{project.name}`.

• A component should return one parent element (or a Fragment).

Props flow down
A parent decides what a child receives. A child should not reach into another component's local state. When a child needs an action, the parent can pass a callback prop.

4. Lists and state in the dashboard

The dashboard keeps remote project data and UI choices in state. When state changes, React renders the dashboard again.

IN YOUR PROJECT: **frontend/src/pages/Dashboard.jsx** - owns project lists, selected tab, loading flags, errors, and create form state.

```
const [myProjects, setMyProjects] = useState([]);
const [allProjects, setAllProjects] = useState([]);
const [tab, setTab] = useState("mine");
const [showCreate, setShowCreate] = useState(false);
const [loading, setLoading] = useState(true);
```

useState returns two things: the current value and a setter function. Calling a setter schedules a render with the new value. Do not change arrays or objects in-place; create a new value instead.

Mapping data into repeated components

```
{myProjects.map((project) => (
  <ProjectCard
    key={project.id}
    project={project}
    isMember={true}
    onDeleted={loadProjects}
  />
))}
```

• **map** turns each project object into a card.

• **key** gives React a stable identity for each rendered item. Use a database id when available.

• The same component is reused for every project; each instance receives different props.

Derived values should usually be calculated

Dashboard calculates which public projects are not already joined by comparing IDs. This is derived UI data: compute it from the two lists instead of maintaining a third copy that can become stale.

Useful question
If a number or list can be calculated from existing props/state, can you compute it during render rather than storing another state variable?

5. Controlled forms and events

A controlled input gets its displayed value from React state. Typing fires onChange, which updates state; submitting reads that state.

IN YOUR PROJECT: **frontend/src/pages/Login.jsx** - uses state for form fields and submits login asynchronously.

```
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");

<input value={email}
  onChange={(event) => setEmail(event.target.value)} />

const handleSubmit = async (event) => {
  event.preventDefault();
  const result = await login(email, password);
  loginSuccess(result);
};
```

Event handling checklist

• Use `onClick` for buttons and `onSubmit` for forms.

• Call `event.preventDefault()` in a form submit handler to stop the browser's default page reload.

• Async handlers use `await` to wait for API responses.

• Show loading or error state so the user can tell whether the action worked.

TaskModal.jsx uses the same pattern for task title, description, assignee, and deadline. TaskRequestModal.jsx conditionally displays status or deadline inputs based on its type state.

Mini exercise
In TaskModal.jsx, find one input. Identify (1) its value, (2) the state variable, and (3) the onChange setter. Follow the same pattern before adding a new input.

6. Follow one API request

Keep interface code separate from network details. TaskSyncer has API helper modules so pages can ask for actions using named functions.

IN YOUR PROJECT: **frontend/src/api/projectApi.js** - exports functions such as getMyProjects and createProject.

IN YOUR PROJECT: **frontend/src/api/axios.js** - creates the shared Axios client, adds authentication headers, and handles expired sessions.

Button click	Page handler	API helper	Axios request
User action	calls helper	chooses endpoint	sends token + data
Screen updates	setter stores result	returns response	FastAPI + database

Example: create a project

• Dashboard form submit handler validates the name.

• It calls `createProject(...)` from projectApi.js.

• The helper calls the shared Axios client, which chooses the API base URL and attaches a bearer token when present.

• When the response returns, the page refreshes its list and closes the form. React renders the updated projects.

Frontend checks are not security
Hiding an edit button improves the interface, but a user can still call an endpoint directly. The backend must enforce who may create, edit, or delete each record.

7. Effects: loading, timers, and cleanup

useEffect is for synchronizing a component with something outside React. In Dashboard, effects load data and refresh it periodically.

IN YOUR PROJECT: **frontend/src/pages/Dashboard.jsx** - uses useEffect for initial project loading and periodic refresh.

```
useEffect(() => {
  loadProjects();
  const timer = setInterval(loadProjects, 30000);
  return () => clearInterval(timer);
}, [loadProjects]);
```

• The effect runs after React renders the component.

• The dependency array says which changing values should cause the effect to run again.

• The returned function is cleanup. It clears the timer when the component is removed or the effect restarts.

• A missing or unstable dependency can cause stale values or repeated calls.

Async data loading

The dashboard requests the two project lists together with `Promise.allSettled`. That means one failed request does not automatically discard the result of the other. Loading and error state let the page render useful feedback.

ProjectView reads the projectId from the URL and loads project details and tasks. When the id changes, the effect should request data for the new project. Its subscription and WebSocket listeners also need cleanup when the page changes or unmounts.

When not to use an effect
Do not use an effect just to calculate a label from state. Calculate it during render. Use an effect when you connect to an external system, such as a server, timer, or event subscription.

8. Shared state with Context

Some information is needed in many distant components. TaskSyncer uses Context for the signed-in user and WebSocket connection management.

IN YOUR PROJECT: **frontend/src/context/AuthContext.jsx** - provides user, loginSuccess, and logout to components below AuthProvider.

```
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const saved = localStorage.getItem("user");
    return saved ? JSON.parse(saved) : null;
  });

  const value = { user, loginSuccess, logout };
  return <AuthContext.Provider value={value}>
    {children}
  </AuthContext.Provider>;
}

export const useAuth = () => useContext(AuthContext);
```

AuthProvider wraps the routed app, so Navbar, ProjectCard, Login, and protected routes can access the same user data through `useAuth()` without threading a user prop through every layer.

Context is for shared values, not every value

• A modal's open/closed state usually belongs in the page that opens it.

• The signed-in user is shared across many pages, so Context fits.

• WebSocketContext owns connection subscriptions and broadcasts messages to subscribers.

Storage note
localStorage survives a browser restart. That explains why a previous session can still be visible after restarting the frontend server. Expired tokens must be handled by the app's authentication flow.

9. Parent and child cooperation

A parent owns data; children receive data and callbacks. This keeps updates in one predictable place.

IN YOUR PROJECT: **frontend/src/components/TaskCard.jsx** - receives one task and callbacks to request or perform actions.

IN YOUR PROJECT: **frontend/src/components/TaskRequestModal.jsx** - collects a member request and calls its onSubmit prop.

```
// Parent renders a child and passes an action.
<TaskRequestModal
  task={selectedTask}
  onSubmit={handleTaskRequest}
  onClose={() => setShowRequest(false)}
/>

// Child reports form data upward.
onSubmit({
  request_type: type,
  requested_status: requestedStatus,
  message: message.trim() || null,
});
```

The modal does not need to know which API endpoint the parent uses. It gathers the form values and calls onSubmit. The page handler can then call the API helper, show a result, and refresh the task data.

This is a common React pattern

Data flows down as props. Events flow upward by calling a callback passed as a prop. For shared app-wide data, Context can avoid passing props through many intermediate components.

Understand the permission boundary
The interface may show different buttons for an owner and a member. Treat those checks as presentation. The API must still verify the user's role before changing task status, priority, deadlines, or project membership.

10. Real-time updates

HTTP loads a snapshot. A WebSocket can deliver updates while the page is open, such as a task update or project deletion.

IN YOUR PROJECT: **frontend/src/context/WebSocketContext.jsx** - manages sockets and broadcasts incoming messages to subscribed components.

```
// Conceptual lifecycle used by the context:
subscribe(projectId, onMessage);
// incoming socket message -> registered listeners
unsubscribe(projectId, onMessage);
```

The context owns connection details so every page does not build its own socket. ProjectView subscribes to relevant project messages and updates its data when it receives them. Unsubscribing matters: forgotten listeners can keep acting after a page is gone or deliver duplicate updates.

Keep realtime and ordinary loading distinct

• An initial GET request fills the page when it opens.

• A WebSocket message communicates changes after the connection is established.

• After important mutations, refetching can reconcile the UI with the server's saved state.

• When testing another device, API and WebSocket URLs must use a host that device can reach; localhost means that device itself.

Debugging clue
If a change appears only after you leave and re-enter a page, check whether the mutation response, HTTP refresh, or WebSocket message is updating the state used by the rendered component.

11. Trace a task status request

Use this as a practice walkthrough of how multiple React concepts work together in TaskSyncer.

1	A member opens a task card. TaskCard receives the task and renders a request action based on its props.
2	Clicking it causes ProjectView to open TaskRequestModal and pass the selected task.
3	TaskRequestModal keeps its form fields in useState and controls the inputs.
4	Submitting prevents the browser reload and calls the onSubmit prop with request data.
5	ProjectView's handler calls a task request API helper and updates feedback or reloads tasks.
6	Axios sends the request with authentication. The backend validates identity, project membership, and permissions.
7	The owner may later approve or reject. On success, state refresh or a WebSocket event updates both views.

How to use this trace

- Start at the button and identify its click handler.
- Search for the modal's component name to see where it is rendered.
- Find the onSubmit prop in the modal, then find the handler passed by ProjectView.
- Follow that handler to the API helper. This is how you learn unfamiliar flows without reading every file.

12. Practice: add task search

This small exercise teaches state, controlled inputs, derived values, and list rendering without changing the backend.

Goal

Add a search box to ProjectView so typing filters visible tasks by title. This is a learning exercise; the PDF does not modify your source files.

Step A - Add state

```
const [searchText, setSearchText] = useState("");
```

Put it beside the other ProjectView useState calls. This state belongs to the page because the page owns the task list.

Step B - Add a controlled input

```
<input
  value={searchText}
  onChange={(event) => setSearchText(event.target.value)}
  placeholder="Search tasks"
/>
```

Place the input near the task columns. Every keystroke updates searchText and causes a render.

Step C - Derive filtered tasks

```
const query = searchText.trim().toLowerCase();
const visibleTasks = tasks.filter((task) =>
  task.title.toLowerCase().includes(query)
);
```

Compute this from the current task list. Do not make a second API call for each keystroke.

Step D - Render the filtered list

Where ProjectView maps tasks into columns or cards, use visibleTasks as the source. Check whether grouping by status happens before or after filtering; either order is fine if the displayed result is correct.

Check your work
Empty search shows every task. A partial title shows matching tasks. No match shows empty columns or a helpful empty message. Clearing the input restores all tasks.

13. Common React mistakes to recognize

When the screen behaves unexpectedly, look for one of these patterns before changing several files at once.

State changed but screen did not - Check whether an array/object was mutated in-place. Use a new array/object in the setter.

Request repeats many times - Review the useEffect dependency list and whether a function dependency is recreated each render.

Input snaps back while typing - Check that value and onChange use the same state variable and setter.

Warning about list keys - Add a stable key such as project.id or task.id to the top element returned from map.

Data appears late or disappears - Inspect loading/error state, network response, token, API base URL, and whether the UI refreshed after mutation.

Role button is hidden but API still changes data - Enforce permissions on the backend. UI conditions are not an access control boundary.

A practical learning order

Session	Practice
1	Trace index.js -> App.jsx -> Dashboard. Name the components you encounter.
2	Explain one useState variable and find every place its setter is called.
3	Trace a form submit to an API helper and back to the rendered list.
4	Study one useEffect and explain its dependencies and cleanup.
5	Complete the task search exercise, then inspect the browser console and network panel.
6	Trace Context and WebSocket updates. Draw the flow in your own notes.

Source map

App startup: **frontend/src/index.js** | Routing: **frontend/src/App.jsx** | Dashboard: **frontend/src/pages/Dashboard.jsx** | Project board: **frontend/src/pages/ProjectView.jsx** | Auth: **frontend/src/context/AuthContext.jsx** | Realtime: **frontend/src/context/WebSocketContext.jsx** | Network: **frontend/src/api/**

You have the map
When you add a feature, identify who owns the data, what event starts the change, where the API call belongs, and what state should change after success.